

AI휴먼 day28

AI 웹서비스 개발 및 배포

데이터·음성 대시보드

오늘의 학습 목표

- Dashboard의 목적과 정보 구조를 설명할 수 있음
- Pandas DataFrame을 Streamlit 화면에 연결할 수 있음
- CSV 파일을 업로드하고 데이터를 확인할 수 있음
- Sidebar 입력값으로 데이터를 필터링할 수 있음
- Metric으로 핵심 지표를 구성할 수 있음
- 데이터에 적합한 차트를 선택해 시각화할 수 있음
- WAV 파일을 업로드하고 웹에서 재생할 수 있음
- 데이터와 음성 기능을 Tabs로 구성할 수 있음



Dashboard란?

Dashboard

- 중요한 정보를 한 화면에서 확인하도록 구성한 시각적 인터페이스
- 데이터의 현재 상태, 핵심 지표, 변화 추이를 빠르게 파악하는 목적

자동차 계기판과 유사

- 속도
- 연료량
- 주행거리
- 경고 상태

데이터 대시보드

- 데이터 건수
- 평균값
- 최대·최소값
- 표
- 차트
- 필터

좋은 대시보드의 정보 구조

입력

- 파일 업로드
- 필터 조건 선택

가공

- 데이터 타입 확인
- 조건에 따른 행 선택
- 집계와 통계 계산

출력

- Metric
- DataFrame
- Chart
- Audio

핵심 흐름 입력 → 가공 → 시각화

오늘 사용할 주요 라이브러리

라이브러리	역할	대표 객체/함수
Streamlit	웹 UI 구성	st.file_uploader(), st.metric()
Pandas	표 형식 데이터 처리	DataFrame, read_csv()
Matplotlib	세부 차트 작성	pyplot, plot(), bar()

Streamlit 자체 차트와 Matplotlib 차트를 함께 활용

Pandas DataFrame

DataFrame

- 행과 열로 구성된 2차원 표 형식 데이터 구조
- Excel 시트나 데이터베이스 테이블과 유사

예)

```
import pandas as pd

df = pd.DataFrame({
 "지역": ["서울", "부산"],
 "매출": [120, 90]
})

st.dataframe(df)
```

Pandas 주요 객체·함수·속성

구분	이름	역할	예
클래스	DataFrame	2차원 표 데이터	pd.DataFrame(data)
함수	read_csv()	CSV 읽기	pd.read_csv(file)
속성	shape	행·열 개수	df.shape
속성	columns	컬럼명	df.columns
속성	dtypes	컬럼 자료형	df.dtypes
메서드	head()	앞부분 확인	df.head()
메서드	describe()	기술통계	df.describe()
메서드	select_dtypes()	타입별 컬럼 선택	df.select_dtypes("number")

Streamlit 데이터 출력

st.dataframe(df)

- 대화형 표
- 스크롤과 컬럼 확인에 적합

st.table(df)

- 정적인 표
- 작은 결과를 고정 형태로 표시할 때 적합

대시보드에서는 데이터가 많을 수 있으므로 st.dataframe()을 주로 사용

파일 업로드

```
uploaded = st.file_uploader(
    "CSV 파일 선택",
    type=["csv"]
)
```

file_uploader()

- 사용자의 로컬 파일을 웹 앱으로 전달
- type으로 허용 확장자 제한 가능
- 업로드 전에는 None
- 업로드 후 파일 객체 반환

CSV 파일 읽기

```
uploaded = st.file_uploader(
    "CSV 파일",
    type=["csv"]
)

if uploaded is not None:
    df = pd.read_csv(uploaded)
    st.dataframe(df)
```

중요

- 업로드 여부를 먼저 검사
- 파일 객체를 read_csv()에 전달
- 읽은 결과는 DataFrame

Metric이란?

Metric

- 대시보드에서 핵심 수치를 강조해 보여주는 구성요소
- KPI(Key Performance Indicator)를 표현할 때 유용

```
st.metric(
    "데이터 건수",
    1250
)
```

KPI 예

- 전체 행 수
- 평균 매출
- 최고 점수
- 오류 건수

기초 통계 출력

```
numeric_df = df.select_dtypes(
    include="number"
)

st.dataframe(
    numeric_df.describe()
)
```

describe()

- count: 데이터 개수
- mean: 평균
- std: 표준편차
- min: 최솟값
- 25%, 50%, 75%: 사분위수
- max: 최댓값

필터링이 필요한 이유

전체 데이터를 그대로 보여주는 것보다 사용자가 관심 있는 조건을 선택할 수 있어야 함

예)

- 지역별
- 상품별
- 연도별
- 등급별

흐름

사용자 선택 →

Boolean 조건 생성 →

DataFrame 필터링 →

Metric과 Chart 갱신



selectbox를 이용한 필터

```
regions = ["전체"] + sorted(
 df["지역"].dropna().unique().tolist()
)
region = st.selectbox(
 "지역",
 regions
)
if region != "전체":
 filtered = df[
 df["지역"] == region
 ]
else:
 filtered = df
```

Streamlit 기본 차트

함수	용도	예
st.line_chart()	변화 추이	시간별 매출
st.bar_chart()	항목 비교	지역별 매출
st.area_chart()	누적 변화	사용량 변화
st.scatter_chart()	두 변수 관계	키와 몸무게

간단한 시각화는 별도 Figure 작성 없이 빠르게 구성 가능

line_chart 예

```
chart_df = df[
    ["날짜", "매출"]
].set_index("날짜")

st.line_chart(
    chart_df
)
```

시간 순서에 따른 값의 변화를 확인할 때 적합

날짜 컬럼은 필요에 따라 datetime 타입으로 변환

bar_chart 예

```
summary = (
    df.groupby("지역")["매출"]
    .mean()
)

st.bar_chart(summary)
```

groupby()

같은 그룹의 데이터를 묶음

mean()

그룹별 평균 계산

대시보드에서는 집계 결과를 시각화하는 경우가 많음

Matplotlib 연결

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.bar(
    summary.index,
    summary.values
)
ax.set_title("지역별 평균 매출")

st.pyplot(fig)
```

Streamlit 기본 차트보다 세부 설정이 필요할 때 활용

시각화 선택 기준

선 그래프

시간에 따른 변화

막대 그래프

범주별 크기 비교

산점도

두 수치 변수의 관계

표

정확한 개별 값 확인

Metric

가장 중요한 숫자 강조

목적에 맞는 시각화를 선택해야 함



파일 업로드와 보안

파일 업로드 기능에서는 다음 사항을 고려

- 필요한 확장자만 허용
- 예상하지 못한 파일 형식 처리
- 지나치게 큰 파일 주의
- 파일명만 믿고 처리하지 않기
- 개인정보나 민감정보가 포함된 데이터 주의

실습에서는 CSV와 WAV로 확장자를 제한

WAV 파일 업로드

```
audio_file = st.file_uploader(
    "WAV 파일 선택",
    type=["wav"]
)

if audio_file is not None:
    st.audio(
        audio_file,
        format="audio/wav"
    )
```

음성 프로젝트에서는

업로드 → 재생 → 전처리 → 모델 추론 순서로 확장 가능

Tabs로 기능 분리

```
data_tab, audio_tab = st.tabs(
    ["데이터 분석", "음성"]
)

with data_tab:
    st.write("CSV 대시보드")

with audio_tab:
    st.write("WAV 업로드")
```

서로 다른 기능을 한 페이지에서 논리적으로 분리할 수 있음

대시보드 레이아웃 예

Sidebar

- CSV 업로드
- 필터 조건

본문 상단

- Metric 3개

본문 중단

- 필터링된 DataFrame

본문 하단

- Chart

별도 Tab

- WAV 업로드와 재생

사용 흐름에 따라 화면을 구성하는 것이 중요

빈 데이터 처리

```
if filtered.empty:
    st.warning(
        "조건에 맞는 데이터가 없습니다."
    )
else:
    st.dataframe(filtered)
```

empty 속성

DataFrame에 데이터가 없으면 True

필터 결과가 비어 있을 가능성을 반드시 고려

실습용 데이터 구조

오늘의 기본 데이터

- 날짜
- 지역
- 방문자수
- 평균체류시간
- 만족도

분석 예

- 지역 필터
- 전체 행 수
- 평균 방문자수
- 평균 만족도
- 날짜별 방문자 변화
- 지역별 평균 방문자 비교

CSV를 업로드하지 않아도 샘플 데이터로 앱 구조를 확인하도록 구성

오늘의 종합 실습

데이터 분석 Tab

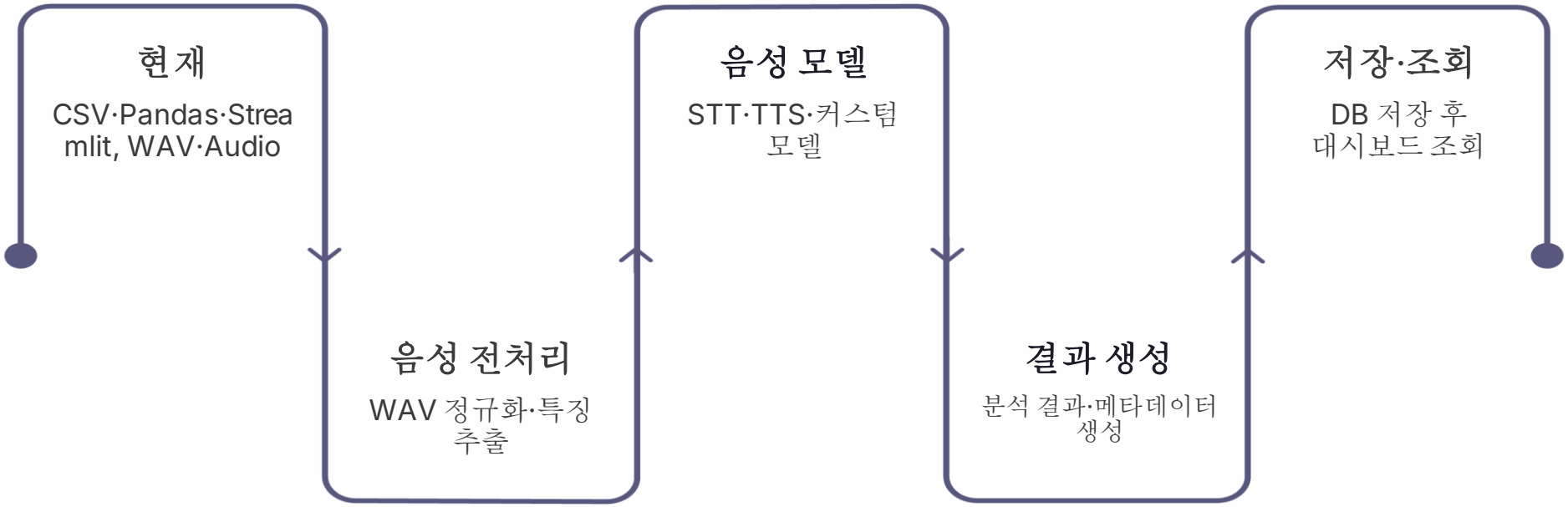
- CSV 업로드 또는 샘플 데이터
- 지역 필터
- Metric 3개
- DataFrame
- 기술통계
- 선 그래프와 막대 그래프

음성 Tab

- WAV 업로드
- 파일 정보
- 음성 재생

핵심 입력 → 가공 → 시각화

프로젝트로 확장하기



현재 CSV → Pandas → Streamlit WAV → Streamlit Audio

향후 WAV → 음성 전처리 → STT/TTS/음성 모델 → 결과 데이터 생성 → DB 저장 → 대시보드 조회

핵심 용어 정리

용어	의미
Dashboard	핵심 정보를 한 화면에 구성한 인터페이스
DataFrame	Pandas의 2차원 표 데이터
Metric	핵심 수치를 강조하는 UI 요소
KPI	핵심 성과 지표
Visualization	데이터를 그래프로 표현하는 과정
File Upload	사용자의 파일을 앱에 전달하는 기능
Filtering	조건에 맞는 데이터만 선택하는 과정
Boolean Indexing	True/False 조건으로 행을 선택하는 방법
Audio	웹에서 음성 데이터를 재생하는 기능

정리

- 대시보드는 입력 → 가공 → 시각화 흐름으로 설계
- CSV는 file_uploader()와 read_csv()로 연결
- DataFrame은 st.dataframe()으로 표시
- Metric은 중요한 숫자를 빠르게 전달
- Widget 값으로 DataFrame을 필터링
- 데이터 특성에 맞는 Chart를 선택
- WAV는 file_uploader()와 st.audio()로 연결
- Tabs로 데이터와 음성 기능을 하나의 앱에 구성
- 다음 단계에서는 데이터를 DB에 저장하는 구조로 확장